

Sistema de Soporte Técnico + Wilmercito (IA local)

Gestión de atenciones de soporte (CRUD + reportes) con asistente de IA local **Wilmercito**: llama.cpp directo en GPU, sin nube y sin Ollama.

Arquitectura (resumen)

Burbuja (Django) · FastAPI/ia/* · Chroma (CPU) + reglas · llama-server (GPU).

| Servicio | Dónde | Puerto | ---|---|---| | llama-server (Qwen2.5-3B Q4_K_M) | host server-mattias (GPU RTX 2060) | 8081 | | backend-fastapi (uvicorn, soporte-api-ia) | VM upds | 5012 | | frontend-django (runserver, soporte-web-ia) | VM upds | 8011 | | PostgreSQL 15 (soporte, soporte_dev) | VM upds | 5432 |

Detalle completo: DOCUMENTACION.md. Bitácora del trabajo con IA docs/bitacora-ia/.

Instalación

Requisitos: Python 3.11+ (3.12 para fine-tuning), PostgreSQL 15, Tailscale entre host y VM.

```
# 1. Base de datos (en la VM)
createdb soporte && psql soporte < backup-plano.sql # SQL plano:
el dump -Fc de PG16 no lo lee PG15
# 2. API
cd backend-fastapi && python3 -m venv .venv && .venv/bin/pip
install -r requirements.txt
cp .env.example .env # completar DATABASE_URL, JWT_SECRET,
SEED_PASSWORD
.venv/bin/alembic upgrade head
# 3. Web
cd frontend-django && python3 -m venv .venv && .venv/bin/pip
install -r requirements.txt
cp .env.example .env # completar DJANGO_SECRET_KEY, FASTAPI_URL,
DJANGO_ALLOWED_HOSTS
.venv/bin/python manage.py migrate
# 4. Motor IA (en el host GPU)
cmake -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=75 ~/llama.cpp &&
cmake --build --target llama-server
bash ~/run-llama.sh # levanta llama-server :8081 con api-key
```

Variables de entorno (LLAMA_URL, LLAMA_API_KEY, LLAMA_TIMEOUT, EMB_MODEL, CHROMA_DIR, RERANK) se leen con valores por defecto en código; ver .env.example de cada proyecto.

Ejecución

```
bash scripts/arrancar_produccion.sh # local: API :5002 + web
:8001 (puertos de desarrollo)
# En la VM productiva gobierna systemd: soporte-api-ia (:5012) +
soporte-web-ia (:8011)
```

Abrir la web · iniciar sesión · **Wilmercito** (abajo-derecha) · preguntar a Wilmercito índice semántico POST /api/ia/reindexar (idempotente). Salud: GET /api/ia/estado.

Pruebas

```

cd backend-fastapi
DATABASE_URL='postgresql+psycopg://x:x@localhost:5433/soporte_test'
\
    .venv/bin/python -m pytest tests/test_ia_unit.py -q    # 13
passed, sin GPU ni DB real

```

Peculiaridad: `tests/conftest.py` exige que la base termine en `_test` y la rechaza en caso contrario. La URL ficticia solo satisface esa guarda; la suite de IA no abre conexión. La batería vivió POST /api/ia/evaluar, 10 casos) sí requiere DB e índice.

Estructura

```

backend-fastapi/app/
  routers/      atenciones.py auth.py ia.py usuarios.py
jerarquia.py areas.py horarios.py
  services/     ia_retrieval.py (embeddings+Chroma) ia_tools.py
(registro tools) categorias.py ...
  models/      atencion.py usuario.py feedback_ia.py
propuesta_ia.py log_ia.py ...
  mcp_soporte/ servidor MCP con 18 tools
frontend-django/
  atenciones/   views.py (CRUD, reportes_vista, wilmercito_vista)
urls.py
  templates/atenciones/ _wilmercito.html (burbuja) reportes.html
asistente.html
scripts/       arrancar_produccion.sh indexar_historico.py ft/
(fine-tuning)
docs/          integracion-llamacpp.md bitacora-ia/ (15 slices)
guia-estudio-predefensa.md

```

Notas honestas

- Django usado: **4.2.16** (el pin 6.1.1 no existe en PyPI; ver docs/bitacora-ia/slice-02).
- El fine-tuning QLoRA dio **5/30** tres noches seguidas: el gate frenó su despliegue.
- Sin Ollama en código ni en servicios `ollama.service` detenido y deshabilitado en el host.